

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение

высшего образования

«Национальный исследовательский университет ИТМО»

(Университет ИТМО)

Факультет программной инженерии и компьютерной техники

Образовательная программа: Системное и прикладное программное
обеспечение

Направление подготовки: Программная инженерия 09.04.04

Дисциплина: Системное программное обеспечение

Практическое задание №1

Обучающийся: Пименов Глеб Андреевич 506758

Группа: Р4115

Дата 20.06.2026

Санкт-Петербург

2026

Оглавление

1. Цели	2
2. Задачи	3
3. Описание работы	5
4. Аспекты реализации	6
Результаты	7
Выводы	7

1. Цели

Используя комплекс программ, разработанный в первом семестре изучения дисциплины Системное программное обеспечение, реализовать алгоритмы планирования задач в многопоточной среде по варианту.

Реализовать тестовую программу и показать различия в ходе её выполнения при запуске под управлением различных алгоритмов планирования.

2. Задачи

1 Изучить порядок работы с таймером в среде выполнения программы (учебной или реальной VM).

2 Реализовать процедуры, необходимые для порождения и переключения контекста CPU между различными логическими потоками управления.

3 Изучить и реализовать заданные алгоритмы планирования задач, используя процедуры из п.2 и таймер в качестве триггера для вызова планировщика.

4 Реализовать тестовую программу, эмулирующую вычислительную нагрузку с помощью заданного алгоритма.

5 Запустить тестовую программу под управлением различных алгоритмов планирования задач и собрать

информацию о ходе её работы.

6 Подготовить отчёт, содержащий результаты выполнения пунктов по порядку.

7 Согласовать содержание отчёта с преподавателем, при необходимости внести правки.

Вариант 14

В соответствии с вариантом задания реализовать алгоритм планирования (2 шт.).

Программно подготовить исходные данные, представляющие собой пары чисел, где первое число – время поступления процесса, а второе – время его выполнения. Очевидно, что последовательность первых значений – возрастающая (временные парадоксы запрещены).

Например:

время появления: 0 4 6 7 11 13 19 24

длительность выполнения 4 7 8 4 5 8 2 5

Диапазон времен выполнения, и среднее время между поступающими процессами взять из варианта задания.

Число пар <время поступления> – <длительность выполнения> не менее двадцати.

Для полученных данных определить среднее время оборота процессов и среднее время ожидания

процессов, в соответствии с реализованными алгоритмами планирования.

Сделать вывод о предпочтительном алгоритме планирования, считая критерием эффективности минимальное среднее время ожидания процессов

Вариант 14:

Алгоритм 1 - RR(2)

Круговое, карусельное, Round Robin (RR)

Модификацией алгоритма FCFS является алгоритм, получивший название Round Robin (Round Robin - это вид детской карусели в США) или сокращенно RR. По сути дела, это тот же самый алгоритм, только реализованный в режиме вытесняющего планирования. Можно представить себе все множество готовых процессов организованным циклически – процессы сидят на карусели. Карусель вращается так, что каждый процесс находится около процессора небольшой фиксированный квант времени, обычно 10 - 100 миллисекунд. Пока процесс находится рядом с процессором, он получает процессор в свое распоряжение и может выполняться. Реализуется такой алгоритм так же, как и предыдущий, с помощью организации процессов, находящихся в состоянии готовности, в очередь FIFO. Планировщик выбирает для очередного исполнения процесс, расположенный в начале очереди, и устанавливает таймер для генерации прерывания по истечении определенного кванта времени. При выполнении процесса возможны два варианта. Время непрерывного использования процессора, необходимое процессу, меньше или равно продолжительности кванта времени. Тогда процесс по своей воле освобождает процессор до истечения кванта времени, на исполнение поступает новый процесс из начала очереди, и таймер начинает отсчет кванта заново. Продолжительность остатка текущего CPU процесса больше, чем квант времени. Тогда по истечении этого кванта процесс прерывается таймером и помещается в конец очереди процессов, готовых к исполнению, а процессор выделяется для использования процессу, находящемуся в ее начале.

Алгоритм 2 - SRT

Shortest remaining time (SRT)

SRT – вытесняющий вариант планирования. В этом варианте планирования планировщик выбирает процесс с наименьшим остающимся временем. Учитывается появление новых процессов в очереди готовых к

исполнению (из числа вновь родившихся или разблокированных) во время работы выбранного процесса.

Если время работы вновь появившегося процесса меньше, чем остаток времени выполнения исполняющегося, то исполняющийся процесс вытесняется новым.

См. Столлинкс В. Операционные системы. Внутреннее устройство и реализация. Часть 4.

Диапазон длительностей выполнения 3-11

Среднее время между поступающими процессами 7, 3

3. Описание работы

Работа состоит в реализации процесса планирования потоков и измерения его качества на уровне виртуальной машины. Основная задача - сделать работу двух алгоритмов планирования в одном проекте и сравнить их по метрикам ожидания и времени выполнения. Система запускает тестовый сценарий, имитирует потоковую среду, выполняет диспетчеризацию и собирает результаты для RR(2) и SRT.

4. Аспекты реализации

scheduler.asm

реализованы два алгоритма: run_rr и run_srt

run_rr использует квант времени и циклическое распределение потоков

run_srt выбирает поток с наименьшим оставшимся временем и может вытеснять текущий поток

Таймер и переключение контекста

vm_runtime.asm содержит логику обработки тика и контекстного переключения

используется threadInterruptPoint для проверки истечения квантов

setThreadQuantumTicks задаёт длину квантового интервала
переключение выполняется на основе сравнения текущего тика и начала кванта

Метрики

сбор выполняется в compute_metrics
вычисляются scheduler_get_avg_wait_x100 и
scheduler_get_avg_turnaround_x100

данные собираются для каждого алгоритма отдельно и выводятся в
итоговом отчёте

Проверка

в vm_two_threads.asm реализован сценарий запуска наборов процессов
выводятся значения RR и SRT, а также выбирается предпочтительный
алгоритм на основе метрик

Результаты

Создана рабочая система планирования в spro2_lab1, включающая:

scheduler.asm с реализацией run_rr и run_srt
vm_runtime.asm с таймерным переключением контекста и обработкой
тиков

vm_two_threads.asm с тестовым сценарием и выводом метрик

Реализованы алгоритмы:

RR(2) - циклический планировщик с квантами

SRT - вытесняющий выбор процесса с минимальным оставшимся
временем

Добавлен сбор метрик:

avg_wait (среднее время ожидания)

avg_turnaround (среднее время оборота)

вывод в формате *_X100 для удобства сравнения

Проведены тесты:

запущен сценарий `vm_two_threads.asm` на 20 процессах
проверка вывода метрик RR и SRT

Количественные оценки:

RR_WAIT_X100=0305

RR_TAT_X100=1000

SRT_WAIT_X100=0190

SRT_TAT_X100=0885

Выводы

Реализация получилась как единая система планирования внутри виртуальной машины: одно ядро для таймерного переключения и сбора данных, два алгоритма диспетчеризации, отдельный запуск тестовых процессов и вывод метрик. В итоге показано, что RR(2) и SRT работают в одной архитектуре, но SRT дает лучшие результаты по среднему времени ожидания и среднему времени оборота для данного набора задач. То есть здесь SRT считается лучшим алгоритмом.